MERN Backend Assessment

API Consumption, JWT Authentication, and Data Handling

1. Objective

This assessment evaluates your understanding of:

- REST API consumption using Axios
- Async/Await and Promise handling
- JWT-based authentication
- Request headers and body handling
- Data caching in memory
- Express routing
- Clean backend architecture
- Deployment to Render

2. Assessment Overview

You will be provided with two external APIs:

1. **Public API** – Issues a JWT token when provided with your student ID
2. **Private API** – Returns dataset only when a valid JWT token is sent in the Authorization header

Your task is to:

1. Request a token using your student ID
2. Use the token to access the private API
3. Fetch the dataset once at server startup
4. Cache the dataset in memory
5. Build five custom endpoints using the cached data
6. Deploy your server on Render
7. Submit both Render and GitHub URLs

3. Step 1 – Obtain JWT Token (Public API)

Endpoint (Provided)

POST /public/token

Request Body

```
{
  "studentId": "YOUR_REGISTER_NUMBER"
}
```

Response Format

```
{
  "token": "JWT_TOKEN_STRING"
}
```

Notes:

- The request must be sent using POST.
- The studentId must be sent in the request body.
- Token validity is limited to 5 minutes.
- Invalid studentId will result in 403 Forbidden.

4. Step 2 – Access Private API

Endpoint (Provided)

GET /private/data

Required Header

Authorization: Bearer <JWT_TOKEN>

Response behavior:

- Valid token → Dataset returned
- Invalid or expired token → 401 Unauthorized

5. Implementation Rules

You must follow the rules below strictly:

1. Fetch the dataset once at server startup.
2. Store the dataset in memory.
3. Use the cached dataset for all custom endpoints.

Not allowed:

- Calling the private API inside each route.
- Hardcoding the token.
- Bypassing the public API.
- Sharing tokens between students.

Failure to follow these rules will result in mark deduction.

6. Endpoints to Implement

Using the cached dataset, implement the following five endpoints:

6.1 Get All Records

GET /records

Returns all records from the dataset.

6.2 Get Record by ID

GET /records/:id

Must use route parameters.

6.3 Get Top Performer

GET /records/top

Must determine highest score using reduce or sorting logic.

6.4 Get Average Score

GET /records/average

Must calculate average using reduce.

6.5 Filter Records Above a Given Value

GET /records/filter?min=VALUE

Must use query parameters and filter logic.

7. Technical Requirements

Your project must include:

- Express
- Axios
- Async/Await
- Proper error handling
- Express.json() middleware
- Clean folder structure (recommended: routes, controllers, services)

Recommended architecture:

routes/

controllers/

services/

app.js

server.js

8. Deployment Requirements

You must:

1. Deploy your project on Render.
2. Ensure application uses process.env.PORT.
3. Ensure the server starts without manual intervention.
4. Submit:
    - GitHub repository URL
    - Render live URL

9. Evaluation Criteria (100 Marks)

Criteria	Marks
Public API call (token retrieval)	10
JWT handling and header usage	15
Private API integration	15
Correct implementation of 5 endpoints	25
Caching implementation	10
Code structure and clarity	10
Error handling	10
Deployment success	5

10. Academic Integrity

- Each student receives a unique token tied to their student ID.
- Tokens expire after 5 minutes.
- Sharing tokens is considered malpractice.
- Any attempt to bypass authentication will result in disqualification.

11. Submission Instructions

Submit the following before the deadline:

1. GitHub repository link
2. Render deployment link

Late submissions will not be accepted.